

Programming and Data Structures Lab Test-II (Section 6)

Department of CSE, IIT Kharagpur

8th April, 2026

General instructions to be followed strictly:

- Name your file as `<roll_no>.<labtest_no>.<question_no>`. For example, if your roll number is 14CS10001 and you are submitting labtest II, question number 1, then name your file as 14CS10001_LTII_1.c. Write your name, roll number, and assignment number at the beginning of your program.
 - Use proper indentation in your code and comment. **Otherwise, marks may be deducted.**
 - Submitting the **correct file** is your responsibility, request of re-submission after the lab will **not** be entertained.
 - **Zero tolerance** policy will be strictly followed for **plagiarism**. The marks will be put straight zero in such cases.
-

Question-1 (15 points)

Define a structure `country` with the following members. Assume the country name and capital will be a string without any whitespace.

- `name` - a character array of size 50 to store the country name
- `capital` - a character array of size 50 to store the country's capital
- `area` - a float to store the area of the country in units of 1,000 sq. km.

Write the following functions:

- (i) `print_country_info( )`: A function that takes an array of `country` structures as argument and prints out the information of all countries. For a particular country, all three fields must be printed in the same line, separated by a pipe character '|'. Every record should be printed on a separate line.
- (ii) `count_smaller_countries( )`: A function that takes an array of `country` structures and a real number `A` as arguments (area in 1000 sq. km) and counts and returns the number of countries in the array that have an area less than `A`.
- (iii) `find_capital( )`: A function that takes an array of `country` structures and the name of a country (a string input by the user) as arguments, and prints the capital of that country. The function should check whether the country entered by the user appears in the array; if yes, its capital should be printed out; if not, the error message "Unknown country" should be printed.

Write a `main( )` function that demonstrates the working of all the above functions. You should ask the user for the number `N` of countries and **dynamically allocate memory** for an array of `N` countries. The `main( )` function should take inputs for all countries from the user. The `main( )` should then print out the records of all countries (using `print_country_info()`). It should ask for an area `A` as input, print out the number of countries having area less than `A`. Also demonstrate the working of the `find_capital( )` function appropriately. You can use the following data to test your program:

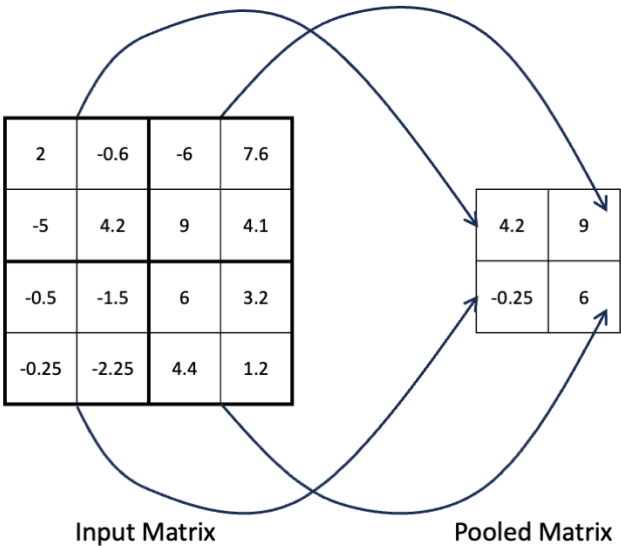
India Delhi 32867.26; Pakistan Islamabad 881.91; Nepal Kathmandu 147.18; Bangladesh Dhaka 147.57;
Germany Berlin 349.22

Question-2 (20 points)

Max-pooling is an important operation in machine learning. The input to this operation is a 2-D square matrix. Every 2×2 submatrix of the input matrix is considered for this purpose. Out of the 4 values in such a 2×2 submatrix, the maximum value is taken and stored in the output matrix. As shown, in the figure below, the maximum value of the 2×2 submatrix (consisting of elements (0,0), (0,1), (1,0) and (1,1)) is 4.2. This value is stored as the (0,0)th element of the Pooled Matrix. As shown, the (0,1)th element of the Pooled Matrix is the maximum of the elements (0,2), (0,3), (1,2) and (1,3) of the Input Matrix.

For the 4×4 Input matrix, naturally, there is not a place to go further right. So, the next max operation takes place between elements (2,0), (2,1), (3,0), (3,1) of the input matrix. Next operation considers the elements (2,2), (2,3), (3,2) and (3,3) of the Input Matrix and stores the maximum value as the (1,1)th element of the Pooled matrix. Note that the input matrix is not necessarily 4×4 always. In general, we will assume a square matrix whose number of rows or columns is a multiple of 2. Naturally, if the input matrix is $n \times n$, the output matrix will be of size $\frac{n}{2} \times \frac{n}{2}$.

Write a C program that should define a square matrix (that is, a 2-D array) in the `main( )` function. You should ask the user to enter the number of rows (or columns) of the matrix. You can take the maximum number of rows or columns to be 20 and assume that the user enters this number as a multiple of 2. Next the user should enter the elements of the square matrix. Your program should compute the pooled matrix as described above and store it in another 2-D array. Your program should print the input as well as the pooled matrix nicely formatted. Use 2 digits after decimal as the precision value for printing. An example output after the user inputs the input matrix can be:



The original matrix:

2.00	-0.60	-6.00	7.60
-5.00	4.20	9.00	4.10
-0.50	-1.50	6.00	3.20
-0.25	-2.25	4.40	1.20

The pooled matrix:

4.20	9.00
-0.25	6.00

Question-3 (15 points)

The field of bioinformatics relies heavily on analyzing DNA, which is represented as a long string of the four chemical bases: Adenine (A), Thymine (T), Cytosine (C), and Guanine (G). A fundamental task in this field is identifying specific genetic sequences, known as a **Target Signature**, within a much larger **Genetic String**. Because DNA can undergo small changes over time, a match is not always perfect. A single difference, called a *point mutation*, occurs when one base is swapped for another.

Your task is to write a C program that performs a search for a **Target Signature** within a larger **Genetic String** while allowing for a **maximum of one mismatch**. The program must first validate that both input strings contain only the valid characters A, T, C, and G. To accomplish this, your code should use a *sliding window* approach, which involves “sliding” a fixed-length string (the size of the signature) over the genetic string one character at a time. For every position, you compare the segment currently inside the window to the signature, ensuring that overlapping sequences are checked. For every valid match found, the program should output the starting index and specify whether zero or one mutation was identified.

For example, if a user provides the Genetic String **GACTGA** and the Target Signature **ACTC**, the program will first compare the window starting at index 0, which is **GACT**. This segment contains two differences compared to the signature, so it is rejected. Moving to index 1, the window “slides” right to compare **ACTG** with **ACTC**. In this case, the first three characters match perfectly, and only the final G differs from the C in the signature. Because this results in exactly one mutation, the program identifies this as a valid match and reports it accordingly before moving to the next position. An example run can be:

(Input) Enter Genetic String: ATCGATCA Enter Target Signature: ATCG	(Output) Match found at index 0 (0 mutations) Match found at index 4 (1 mutation) Total matches found: 2
--	--

The questions end here.

The next few pages are added extra. You can use these, if for any question you need extra space.
